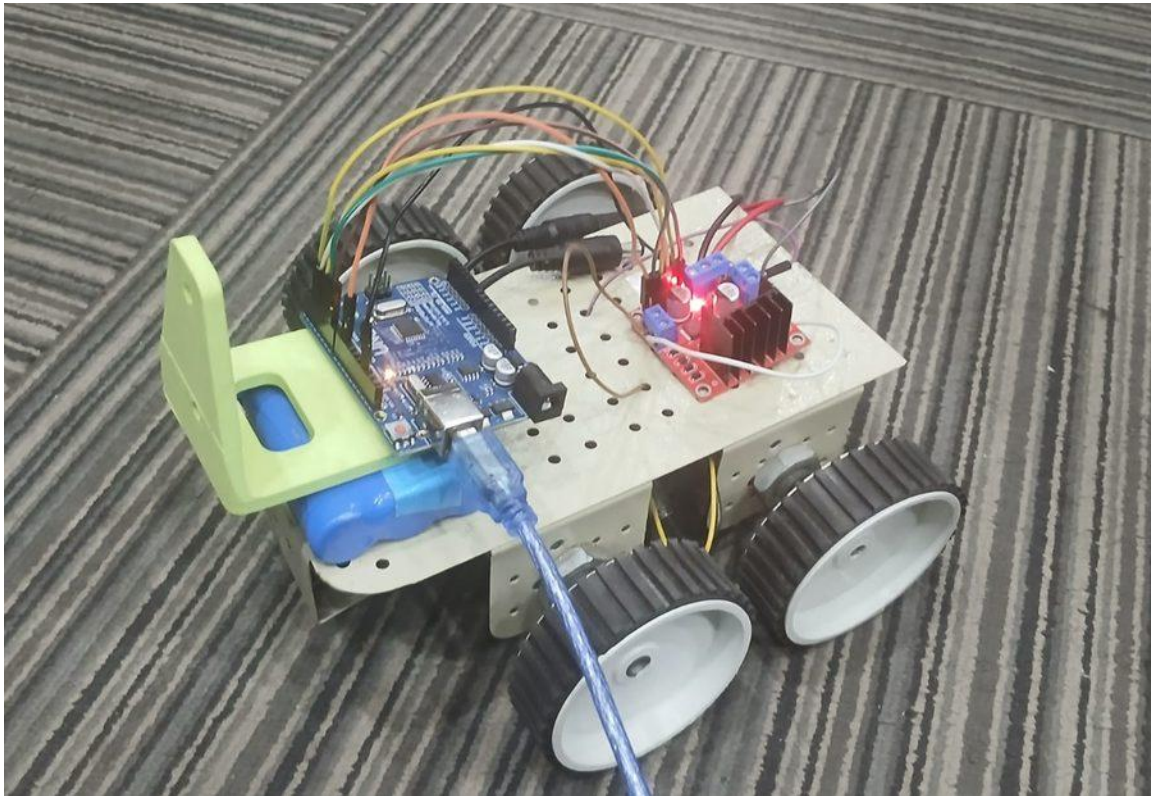


BEGINNER GUIDE TO JOYSTICK CONTROL FOR RC CAR

KY-023 Joystick | Forward Backward Left Right | Horn on Center Press | Step by Step



Your RC Car — now getting physical joystick control with KY-023 module

What You Will Be Able to Do After This Guide

Push joystick UP = Car goes Forward | Push DOWN = Backward | Push LEFT = Turn Left | Push RIGHT = Turn Right
Press joystick CENTER button = Car horn (buzzer beeps) | Release joystick = Car stops

Phase 1 — Understanding the KY-023 Joystick

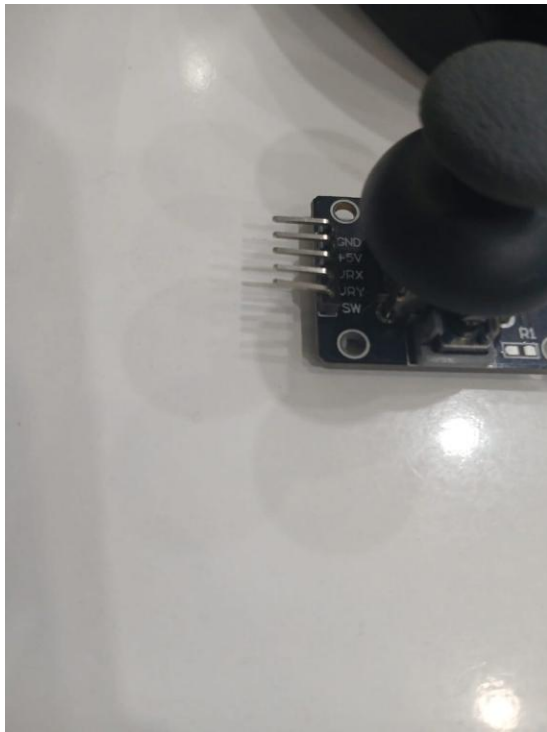


Fig 1 — Your KY-023 joystick: pins GND +5V VRX VRY SW visible



Fig 2 — Side view showing joystick mechanism and two potentiometers



Fig 3 — Top view: the joystick capsule sits on two potentiometers for X and Y axes

1.1 What is the KY-023 Joystick Module?

Your module (shown in the photos) is a KY-023 dual-axis joystick. It is exactly the same type of joystick used in PlayStation and Xbox game controllers — just smaller and without the shell.

Pin on Your Module	Full Name	Type	What It Measures
GND	Ground	Power	Connect to Arduino GND
+5V	Power	Power	Connect to Arduino 5V
VRX	Variable Resistor X	Analog 0-1023	Left-Right movement (X axis)
VRY	Variable Resistor Y	Analog 0-1023	Up-Down movement (Y axis)
SW	Switch	Digital 0 or 1	Center button press (0 = pressed)

1.2 How the Joystick Works — Simple Explanation

Think of it as Two Volume Knobs + One Button

VRX is like a volume knob for left/right. Center position = 512. Push left = goes to 0. Push right = goes to 1023.

VRX is like a volume knob for up/down. Center position = 512. Push up = goes to 0. Push down = goes to 1023.

SW is a button hidden inside the joystick shaft. Press the joystick DOWN = button press = LOW (0). Arduino reads these 3 values and decides: forward, backward, left, right, stop, or horn.

Release joystick to center = both VRX and VRY return to ~512 = car stops.

1.3 Joystick Value Map

Physical Action	VRX Value	VRY Value	SW Value	Car Does
No movement (center)	~512	~512	HIGH (1)	Stop
Push UP (forward)	~512	0 to 400	HIGH (1)	Forward
Push DOWN (backward)	~512	624 to 1023	HIGH (1)	Backward
Push LEFT	0 to 400	~512	HIGH (1)	Turn Left
Push RIGHT	624 to 1023	~512	HIGH (1)	Turn Right
Press center button	~512	~512	LOW (0)	HORN beeps

1.4 What is Dead Zone?

The dead zone is the center region where joystick wobble is ignored. Even when you are not touching the joystick, it rarely reads exactly 512 — it might read 508 or 516. Without a dead zone, the car would constantly try to move slightly. We use values 400 to 624 as dead zone — inside this range = treat as center = stop.

Phase 2 — Components Needed

2.1 What You Already Have

Component	Status	Role
Arduino UNO R3	Already on car	Brain — reads joystick, controls motors
L298N Motor Driver	Already wired	Controls motor direction and speed
4x DC Motors	Already working	Spins the wheels
Battery Pack 6-9V	Already connected	Powers motors
USB Cable	Already used	Powers Arduino
Jumper Wires	Already used	Connections

2.2 New Components to ADD

New Component	Qty	Cost India	Purpose
KY-023 Joystick Module	1	Rs 60-100	Your joystick (already have it!)
Buzzer (Active)	1	Rs 20-30	Horn sound when center pressed
Jumper Wires (M-F)	5	Rs 10	Connect joystick to Arduino
Jumper Wire (M-M)	1	Rs 5	Connect buzzer to Arduino

Active Buzzer vs Passive Buzzer

Active buzzer: has a circuit inside, beeps when you give it 5V. Just connect power = sound.

Passive buzzer: needs a frequency signal to make sound (like `tone()` function in Arduino).

For this project: use ACTIVE buzzer. Simpler wiring, works with just `digitalWrite(HIGH)`.

How to identify: active buzzer has a small white sticker on top usually. Both look similar.

If you only have passive buzzer: replace `digitalWrite(BUZZER,HIGH)` with `tone(BUZZER,1000)`.

Phase 3 — Wiring Connections

3.1 Joystick to Arduino

KY-023 Pin	Connect To	Wire Color	Explanation
GND	Arduino GND	Black	Ground connection — mandatory
+5V	Arduino 5V	Red	Powers the joystick module
VRX	Arduino A0	Orange/Yellow	X-axis analog signal (left/right)
VRY	Arduino A1	Green	Y-axis analog signal (up/down)
SW	Arduino Pin 7	White/Blue	Center button (LOW when pressed)

3.2 Buzzer to Arduino

Buzzer Pin	Connect To	Wire Color	Explanation
+ (positive/longer leg)	Arduino Pin 8	Red	Signal pin — HIGH = sound ON
- (negative/shorter leg)	Arduino GND	Black	Ground

3.3 Complete Wiring Summary (All Components)

From	To	Wire	Purpose
Joystick GND	Arduino GND	Black	Joystick ground
Joystick +5V	Arduino 5V	Red	Joystick power
Joystick VRX	Arduino A0	Orange	X-axis: left/right
Joystick VRY	Arduino A1	Green	Y-axis: up/down
Joystick SW	Arduino Pin 7	White	Center button: horn
Buzzer +	Arduino Pin 8	Red	Horn output
Buzzer -	Arduino GND	Black	Buzzer ground
Arduino Pin 2	L298N IN1	Blue	Left motor dir 1 (existing)
Arduino Pin 3	L298N IN2	Blue	Left motor dir 2 (existing)
Arduino Pin 4	L298N IN3	Blue	Right motor dir 1 (existing)
Arduino Pin 5	L298N IN4	Blue	Right motor dir 2 (existing)
Arduino Pin 6	L298N ENA	Blue	Left motor speed (existing)
Arduino Pin 9	L298N ENB	Blue	Right motor speed (existing)
Arduino GND	L298N GND	Black thick	CRITICAL — must exist (existing)
Battery +	L298N VIN	Red thick	Motor power (existing)

From	To	Wire	Purpose
Battery -	L298N GND	Black thick	Battery ground (existing)

Pin Usage Summary — No Conflicts

Arduino A0 = Joystick VRX (new)

Arduino A1 = Joystick VRY (new)

Arduino Pin 7 = Joystick SW button (new)

Arduino Pin 8 = Buzzer (new)

Arduino Pins 2,3,4,5,6,9 = L298N motor driver (existing — keep as is)

Arduino Pins 10,11 = HC-05 Bluetooth (if using BT version)

All existing wires stay exactly the same. Just add 6 new wires.

Phase 4 — Arduino Code: Joystick Only Mode

Use this code when you want to control the car with the joystick only. No Bluetooth. No laptop keyboard. Just the physical joystick. This is the simplest version.

`rc_car_joystick.ino` | Arduino C++ | Upload to Arduino UNO | Joystick + Buzzer only

```
// =====
// RC CAR — JOYSTICK CONTROL CODE
// Hardware : Arduino UNO + L298N + KY-023 Joystick Module
// Joystick : VRX = A0, VRY = A1, SW = Pin 7
// Horn      : Buzzer on Pin 8 (press joystick center = horn)
// =====
// HOW JOYSTICK WORKS:
// Joystick gives analog values 0 to 1023
// Center position = ~512 (middle value)
// Push UP (forward) = VRY goes to 0
// Push DOWN (back) = VRY goes to 1023
// Push LEFT = VRX goes to 0
// Push RIGHT = VRX goes to 1023
// Press center (SW) = SW reads LOW (0) = HORN
// =====

// — MOTOR PINS —
const int IN1=2, IN2=3, IN3=4, IN4=5;
const int ENA=6, ENB=9;
int L_SPEED=190, R_SPEED=200, TURN_SPEED=160;

// — JOYSTICK PINS —
const int JOY_X = A0; // VRX pin - left/right
const int JOY_Y = A1; // VRY pin - forward/backward
const int JOY_SW = 7; // SW pin - center press = horn

// — BUZZER PIN —
const int BUZZER = 8; // Connect buzzer + to pin 8, - to GND

// — JOYSTICK THRESHOLDS —
// Center value is 512. We only move when joystick is pushed enough.
// DEAD_ZONE: values between 400-624 = treat as center (no movement)
// This prevents accidental movement from slight joystick wobble.
const int DEAD_LOW = 400; // Below this = pushed far enough
const int DEAD_HIGH = 624; // Above this = pushed far enough

// — SETUP —
void setup() {
  // Motor pins
  pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
  pinMode(ENA,OUTPUT); pinMode(ENB,OUTPUT);

  // Joystick button (SW) - INPUT_PULLUP means pin reads HIGH normally
  // When button pressed, it connects to GND and reads LOW
  pinMode(JOY_SW, INPUT_PULLUP);

  // Buzzer
```

```

pinMode(BUZZER, OUTPUT);
digitalWrite(BUZZER, LOW);

Serial.begin(9600);
Serial.println("=== JOYSTICK RC CAR READY ===");
Serial.println("Push joystick: UP=Fwd DOWN=Bwd LEFT=Left RIGHT=Right");
Serial.println("Press CENTER = HORN");
Serial.println("Release joystick = Stop");
stopCar();
}

// — MAIN LOOP —————
void loop() {
  // Read joystick values (0 to 1023, center = ~512)
  int x = analogRead(JOY_X); // left=0, center=512, right=1023
  int y = analogRead(JOY_Y); // up=0, center=512, down=1023
  int sw = digitalRead(JOY_SW); // LOW when pressed

  // Print values for debugging (comment out after testing)
  // Serial.print("X:"); Serial.print(x);
  // Serial.print(" Y:"); Serial.print(y);
  // Serial.print(" SW:"); Serial.println(sw);

  // — HORN (center press) —————
  if (sw == LOW) {
    digitalWrite(BUZZER, HIGH); // turn horn ON
    Serial.println(">>> HORN!");
  } else {
    digitalWrite(BUZZER, LOW); // turn horn OFF
  }

  // — MOVEMENT (joystick direction) —————
  // Priority: Forward/Backward first, then Left/Right
  // Y axis controls forward/backward
  // X axis controls left/right

  if (y < DEAD_LOW) {
    // Joystick pushed UP = FORWARD
    moveForward();
  } else if (y > DEAD_HIGH) {
    // Joystick pushed DOWN = BACKWARD
    moveBackward();
  } else if (x < DEAD_LOW) {
    // Joystick pushed LEFT = TURN LEFT
    turnLeft();
  } else if (x > DEAD_HIGH) {
    // Joystick pushed RIGHT = TURN RIGHT
    turnRight();
  } else {
    // Joystick in center = STOP
    stopCar();
  }
}

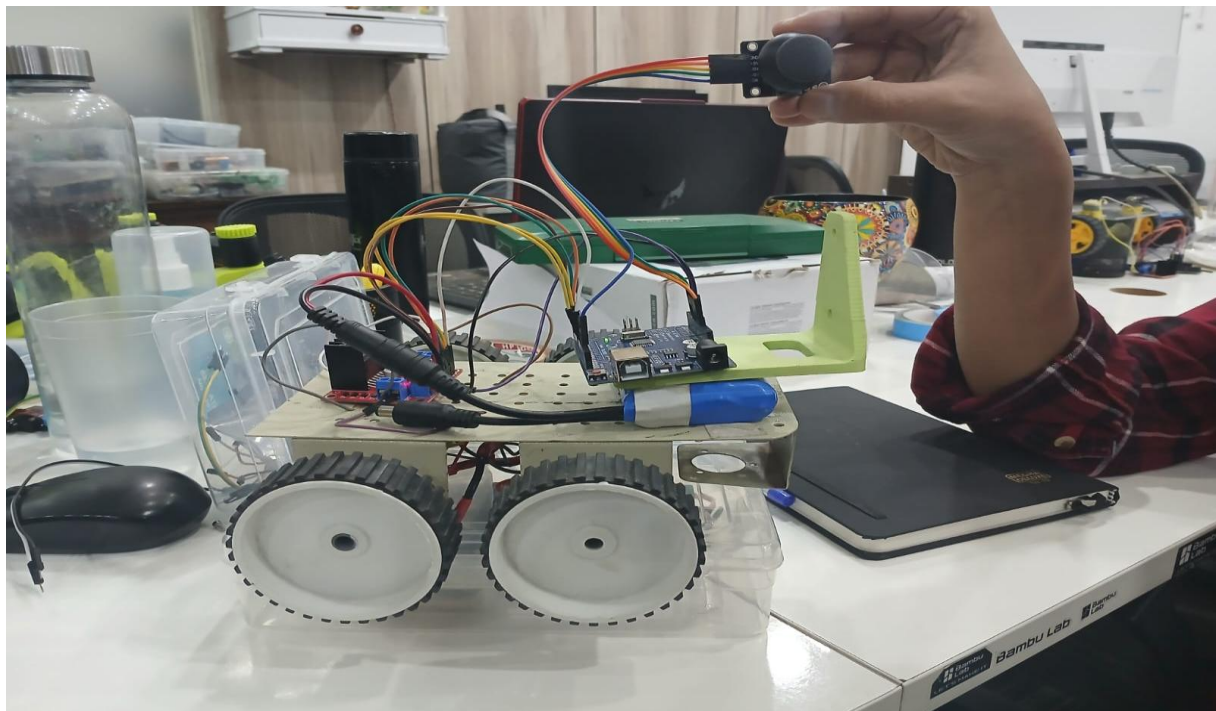
```

```

    delay(50); // Small delay for stability (50ms = 20 readings/sec)
}

// — MOVEMENT FUNCTIONS —————
void moveForward() {
    digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
    digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
    analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED);
}
void moveBackward() {
    digitalWrite(IN1,LOW);  digitalWrite(IN2,HIGH);
    digitalWrite(IN3,LOW);  digitalWrite(IN4,HIGH);
    analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED);
}
void turnLeft() {
    digitalWrite(IN1,LOW);  digitalWrite(IN2,LOW);
    digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
    analogWrite(ENA,0); analogWrite(ENB,TURN_SPEED);
}
void turnRight() {
    digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW);  digitalWrite(IN4,LOW);
    analogWrite(ENA,TURN_SPEED); analogWrite(ENB,0);
}
void stopCar() {
    digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
    analogWrite(ENA,0); analogWrite(ENB,0);
}

```



4.1 Code Explanation — Every Important Part

Code Part	What It Does	Simple Explanation
<code>const int JOY_X = A0</code>	Defines which pin reads X axis	A0 is an analog pin — reads voltage as 0 to 1023
<code>const int JOY_Y = A1</code>	Defines which pin reads Y axis	A1 reads up/down position as number
<code>const int JOY_SW = 7</code>	Defines center button pin	Digital pin 7 reads HIGH or LOW
<code>const int BUZZER = 8</code>	Defines buzzer pin	Pin 8 outputs HIGH or LOW to control horn
<code>INPUT_PULLUP</code>	Makes pin 7 read HIGH normally	Without this, SW pin floats randomly — gives wrong readings
<code>DEAD_LOW = 400</code>	Below this = joystick pushed enough	Values 400-624 are treated as center (no movement)
<code>DEAD_HIGH = 624</code>	Above this = joystick pushed enough	Prevents car moving from tiny joystick wobble
<code>analogRead(JOY_X)</code>	Reads X position as 0-1023	Returns a number: 0=full left, 512=center, 1023=full right
<code>digitalRead(JOY_SW)</code>	Reads button state	Returns LOW(0) when pressed, HIGH(1) when not pressed
<code>if (sw == LOW)</code>	Check if center button pressed	When pressed: sw=LOW, so this condition is true
<code>digitalWrite(BUZZER, HIGH)</code>	Turn buzzer ON	Sends 5V to buzzer = makes sound
<code>if (y < DEAD_LOW)</code>	Check if pushed far enough up	y less than 400 means genuinely pushed up = forward
<code>delay(50)</code>	Wait 50 milliseconds	Reads joystick 20 times per second — smooth and stable

Phase 5 — Arduino Code: Joystick + Bluetooth + Laptop (Triple Mode)

Use this code when you want ALL THREE controls working at the same time: joystick physically on the car, Bluetooth from phone, AND laptop keyboard via Python. Joystick gets priority when used.

How Priority Works

When joystick is pushed: joystick commands run. BT and USB are ignored.

When joystick is at center: BT and USB commands work normally.

When joystick is released from a push: car stops first, then waits for BT/USB.

Horn (SW press) works always — regardless of which other control is active.

Result: you can hand the joystick to someone while you still have laptop control.

rc_car_triple.ino | Arduino C++ | Joystick + HC-05 BT + Laptop USB — all together

```
// =====
// RC CAR — JOYSTICK + BLUETOOTH + LAPTOP TRIPLE CONTROL
// Hardware : Arduino UNO + L298N + KY-023 Joystick + HC-05 BT
// This code handles ALL THREE controls simultaneously:
// 1. Joystick (physical analog control — always active)
// 2. Bluetooth phone (Android app commands)
// 3. Laptop USB Serial (Python arrow keys)
// =====

#include <SoftwareSerial.h>
SoftwareSerial bluetooth(10, 11); // HC-05 TX->Pin10, RX->Pin11

// — MOTOR PINS —————
const int IN1=2, IN2=3, IN3=4, IN4=5;
const int ENA=6, ENB=9;
int L SPEED=190, R SPEED=200, TURN SPEED=160;

// — JOYSTICK PINS —————
const int JOY_X=A0, JOY_Y=A1, JOY_SW=7;
const int BUZZER=8;
const int DEAD LOW=400, DEAD HIGH=624;

// — CONTROL SOURCE TRACKING —————
// Joystick gets priority when it is being used.
// Serial/BT commands work when joystick is at center.
bool joystickActive = false;

void setup() {
  pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
  pinMode(ENA,OUTPUT); pinMode(ENB,OUTPUT);
  pinMode(JOY_SW, INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);
  Serial.begin(9600);
```

```

    bluetooth.begin(9600);
    Serial.println("=== JOYSTICK + BT + LAPTOP MODE ===");
    Serial.println("Joystick: always active (priority)");
    Serial.println("Bluetooth: active when joystick is centered");
    Serial.println("Laptop USB: active when joystick is centered");
    stopCar();
}

void loop() {
    int x = analogRead(JOY_X);
    int y = analogRead(JOY_Y);
    int sw = digitalRead(JOY_SW);

    // Horn from joystick center press
    if (sw == LOW) {
        digitalWrite(BUZZER, HIGH);
        Serial.println(">> HORN!");
    } else {
        digitalWrite(BUZZER, LOW);
    }

    // Check if joystick is being actively pushed
    bool joyPushed = (y < DEAD_LOW || y > DEAD_HIGH || x < DEAD_LOW || x > DEAD_HIGH);

    if (joyPushed) {
        // Joystick is being used - give it full priority
        joystickActive = true;
        if (y < DEAD_LOW) moveForward();
        else if (y > DEAD_HIGH) moveBackward();
        else if (x < DEAD_LOW) turnLeft();
        else if (x > DEAD_HIGH) turnRight();
    } else {
        // Joystick centered - check BT and USB serial
        if (joystickActive) {
            stopCar(); // Joystick just released - stop first
            joystickActive = false;
        }

        char cmd = 0;
        if (Serial.available() > 0) cmd = Serial.read();
        if (bluetooth.available() > 0) cmd = bluetooth.read();
        if (cmd != 0) processCommand(cmd);
    }

    delay(50);
}

void processCommand(char cmd) {
    switch(cmd) {
        case 'F': case 'f': moveForward(); break;
        case 'B': case 'b': moveBackward(); break;
        case 'L': case 'l': turnLeft(); break;
        case 'R': case 'r': turnRight(); break;
        case 'S': case 's': stopCar(); break;
        case 'G': case 'g': rotateLeft(); break;
        case 'I': case 'i': rotateRight(); break;
    }
}

```

```

        case '1': L_SPEED=60; R_SPEED=60; break;
        case '2': L_SPEED=130; R_SPEED=130; break;
        case '3': L_SPEED=190; R_SPEED=200; break;
    }
}

void moveForward(){
    digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
    digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
    analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED);
}
void moveBackward(){
    digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
    digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
    analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED);
}
void turnLeft(){
    digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
    digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
    analogWrite(ENA,0); analogWrite(ENB,TURN_SPEED);
}
void turnRight(){
    digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
    analogWrite(ENA,TURN_SPEED); analogWrite(ENB,0);
}
void stopCar(){
    digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
    analogWrite(ENA,0); analogWrite(ENB,0);
}
void rotateLeft(){
    digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
    digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
    analogWrite(ENA,TURN_SPEED); analogWrite(ENB,TURN_SPEED);
}
void rotateRight(){
    digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
    analogWrite(ENA,TURN_SPEED); analogWrite(ENB,TURN_SPEED);
}
}

```

Phase 6 — Python Calibration Script

This Python script reads the joystick values so you can see if your specific joystick needs different dead zone values. Run this once during setup, note your values, then adjust DEAD_LOW and DEAD_HIGH in Arduino code.

`joy_calibrate.py` | Python 3 | Run once to calibrate joystick values

```
# =====
# RC CAR — PYTHON JOYSTICK MONITOR SCRIPT
# Purpose: Read joystick values from Arduino Serial Monitor
#           so you can calibrate the dead zone
# Install: pip install pyserial
# =====
# HOW TO USE:
# 1. Upload rc_car_joystick.ino to Arduino
# 2. Uncomment the Serial.print lines in loop() in Arduino code
# 3. Run this Python script
# 4. Move joystick in all directions
# 5. Note the X and Y values at extremes
# 6. Update DEAD_LOW and DEAD_HIGH in Arduino code if needed
# 7. Re-comment the Serial.print lines after calibration
# =====
import serial, time, sys

COM_PORT = "COM5"    # Your Arduino USB COM port
BAUD_RATE = 9600

print("Joystick Calibration Monitor")
print("Move joystick in all directions to see values")
print("Center should read around X:512 Y:512")
print("Press Ctrl+C to stop\n")

try:
    arduino = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)
    print("Connected! Reading joystick values...\n")
except:
    print("ERROR: Cannot connect to " + COM_PORT)
    sys.exit(1)

try:
    while True:
        if arduino.in_waiting:
            line = arduino.readline().decode('utf-8','ignore').strip()
            if line:
                print(line)
except KeyboardInterrupt:
    print("\nStopped.")
    arduino.close()
```

6.1 How to Calibrate

1. Upload `rc_car_joystick.ino` to Arduino.

2. In the Arduino code, uncomment these 3 lines in loop():

```
// Serial.print("X:"); Serial.print(x);
```

```
// Serial.print(" Y:"); Serial.print(y);
```

```
// Serial.print(" SW:"); Serial.println(sw);
```

Remove the // from each line to uncomment them. Re-upload.

3. Run: python joy_calibrate.py

4. Move joystick fully UP — note the Y value (should be near 0).

5. Move joystick fully DOWN — note the Y value (should be near 1023).

6. Move joystick fully LEFT — note the X value (should be near 0).

7. Move joystick fully RIGHT — note the X value (should be near 1023).

8. Do NOT touch joystick — note center X and Y values (should be near 512).

9. If center reads 490, set DEAD_LOW=420 and DEAD_HIGH=560 to be safe.

10. After calibration: re-comment the Serial.print lines. Re-upload final code.

Phase 7 — Testing Procedure

7.1 Hardware Check Before Powering On

11. Joystick GND connected to Arduino GND.
12. Joystick +5V connected to Arduino 5V.
13. Joystick VRX connected to Arduino A0.
14. Joystick VRY connected to Arduino A1.
15. Joystick SW connected to Arduino Pin 7.
16. Buzzer + connected to Arduino Pin 8.
17. Buzzer - connected to Arduino GND.
18. All existing motor wires still connected — do not remove any.

7.2 Testing Step by Step

19. Upload rc_car_joystick.ino. Open Serial Monitor at 9600 baud.
20. Verify JOYSTICK RC CAR READY appears.
21. Place car on floor with 50cm space around it.
22. WITHOUT touching joystick: car should stay stopped.
23. Gently push joystick UP. Car should move forward. Release = stop.
24. Push DOWN. Car moves backward. Release = stop.
25. Push LEFT. Car turns left. Release = stop.
26. Push RIGHT. Car turns right. Release = stop.
27. Press joystick DOWN into center. Buzzer should beep. Release = stop.

7.3 Troubleshooting

Problem	Cause	Fix
Car moves without touching joystick	Dead zone too small	Increase DEAD_HIGH or decrease DEAD_LOW in code
Joystick barely responds — needs big push	Dead zone too large	Decrease DEAD_HIGH or increase DEAD_LOW
Forward and backward are swapped	VRY wire direction	Swap DEAD_LOW and DEAD_HIGH conditions in code
Left and right are swapped	VRX wire direction	Swap DEAD_LOW and DEAD_HIGH for X conditions
Horn does not work	Buzzer wiring or passive buzzer	Check buzzer + to Pin 8. If passive: use tone(BUZZER,1000)
SW not working — no horn	Missing INPUT_PULLUP	Verify pinMode(JOY_SW, INPUT_PULLUP) is in setup()
Car drifts in one direction	Motor speed difference	Adjust L_SPEED/R_SPEED constants (lower the faster side)

Problem	Cause	Fix
Joystick values look wrong	VRX and VRY swapped	Swap A0 and A1 pin assignments in code

Phase 8 — Which Code to Use When

Situation	Arduino Code	Extra Script Needed	Notes
Joystick only — simplest	rc_car_joystick.ino	None	Just joystick + buzzer, no laptop needed
Joystick + laptop Python	rc_car_joystick.ino	rc_car_usb.py	Both work; joystick always active
Joystick + BT + Laptop	rc_car_triple.ino	rc_car_bt.py (optional)	Full triple mode, joystick has priority
Calibrate joystick values	rc_car_joystick.ino	joy_calibrate.py	Uncomment Serial.print lines first
Laptop only (no joystick)	rc_car_normal.ino	rc_car_usb.py	Original mode from before
Phone BT only (no joystick)	rc_car_bluetooth.ino	None needed	Use phone app to control

8.1 Upgrade Path Summary

Stage	What You Have	Control Methods
Stage 1 (Done)	Arduino + L298N + motors	Laptop USB keyboard
Stage 2 (Done)	+ HC-05 Bluetooth	Laptop USB + Phone Bluetooth
Stage 3 (This guide)	+ KY-023 Joystick + Buzzer	Laptop + Phone + Physical Joystick + Horn
Stage 4 (Next)	+ HC-SR04 Ultrasonic	All above + Auto obstacle avoidance
Stage 5 (Future)	+ Line sensor	Autonomous line following

Quick Summary — 3 Things to Do Right Now

1. Wire joystick: GND-Black, +5V-Red, VRX-A0, VRY-A1, SW-Pin7
2. Wire buzzer: + to Pin 8, - to GND
3. Upload rc_car_joystick.ino and push joystick UP — car moves forward!
That is all. 3 steps. No Python needed for joystick mode.